

MANIFOLD

A Mathematical Engine for Mapping Vulnerabilities

MANIFOLD Project

Whitepaper v0.1 — working draft, not peer reviewed

Abstract

MANIFOLD treats a software artifact not as a bag of rules to match, but as a *mathematical object*: a typed, weighted graph enriched with *algebraic*, *spectral*, *topological* and *geometric* structure. Over this object we compute a scalar field of **vulnerability potential** $V(x)$, which surfaces anomalous regions as geometric / topological / spectral *features* rather than as literal string matches. An **autonomous LLM agent** navigates the resulting “manifold”, formulates hypotheses about those regions, and dispatches **formal verifiers** (SMT, symbolic execution, abstract interpretation) that confirm or refute each hypothesis — closing the loop between statistical intuition and mathematical proof. This document defines the intermediate representation, formalizes each mathematical layer, states the mapping laws that connect mathematical features to vulnerability classes, and specifies the agent architecture and roadmap.

1 Motivation

Traditional scanners (SAST/DAST) enumerate syntactic patterns. They are brittle: they miss novel vulnerabilities and drown analysts in false positives. Two capabilities are missing:

- (1) A *unified, quantitative representation* of “program structure” onto which many *independent* signals can be projected and fused.
- (2) A *reasoning loop* that converts “suspicious region” into “proven vulnerability” — the LLM supplies the intuition, mathematics supplies the proof.

MANIFOLD provides both.

2 The core idea

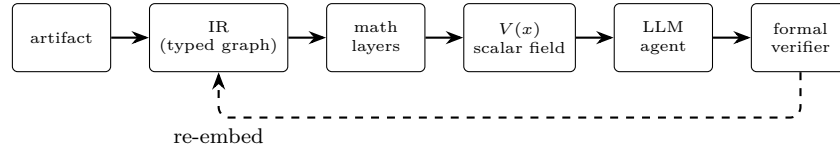


Figure 1: The pipeline: an artifact becomes a typed graph, enriched with mathematical structure into a vulnerability scalar field $V(x)$, which an LLM agent navigates; a formal verifier confirms or refutes each hypothesis and the result re-embeds into the manifold.

A vulnerability is *not* a pattern; it is a *violation of a structural invariant* (a broken naturality condition, a persistent cycle, a negative curvature chokepoint, a taint flow across a spectral trust cut). Each layer detects a different *kind* of invariant violation, and the fused field $V(x)$ ranks regions for the agent to inspect.

3 The Intermediate Representation (IR)

Definition 1 (Program graph). A *program graph* is a tuple $G = (V, E, \tau_V, \tau_E, \omega)$ where

- V is a finite set of *nodes* (modules, functions, blocks, statements, calls, variables, sources, sinks);
- $E \subseteq V \times V$ is a set of directed *edges*;
- $\tau_V : V \rightarrow \mathbb{N}_K$ assigns each node a *kind* from $K = \{\text{module, function, block, statement, call, assign, variable, parameter, source, sink, gate}\}$;
- $\tau_E : E \rightarrow \mathcal{T}$ assigns each edge a *type* $\mathcal{T} = \{\text{control, data, call, taint, trust, auth}\}$;
- $\omega : E \rightarrow \mathbb{R}_{\geq 0}$ assigns each edge a non-negative *weight* (default 1).

The IR is the single source of truth. Every adapter (source code, binary, web API, LLM agent) must produce a program graph; every analysis consumes it. The schema is language-agnostic and JSON-serializable (the Python and Rust layers share the identical schema).

4 Mathematical layers

For each layer we fix a subgraph by edge type τ_E , then compute structure.

4.1 Spectral layer (algebraic graph theory)

For an edge type t , define the symmetric *adjacency* A_t and *degree* D_t , and the *combinatorial Laplacian*

$$L_t = D_t - A_t.$$

L_t is positive semidefinite; its spectrum $0 = \lambda_1 \leq \lambda_2 \leq \dots$ encodes global structure:

- λ_2 is the *algebraic connectivity* (Fiedler value). Its eigenvector f_2 — the *Fiedler vector* — induces a canonical cut $S^+ = \{v : f_2(v) \geq 0\}$.
- The *spectral embedding* $\Phi^{(d)}(v) = (f_2(v), f_3(v), \dots, f_{d+1}(v))$ maps nodes into $\mathbb{R}^d$.
- The Cheeger inequality bounds the bottleneck ratio $h(G)$ by $\frac{\lambda_2}{2} \leq h(G) \leq \sqrt{2 d_{\max} \lambda_2}$.

Signal 1 (spectral anomaly). The reconstruction error of a node under a low-rank eigenmap reconstruction, and its distance from the Fiedler cut, are used as anomaly scores.

4.2 Topological layer (persistent homology)

From the program graph we build a *filtration* of simplicial complexes (e.g. a Vietoris–Rips or flag complex over the shortest-path metric, or a filtration driven by ω). Persistent homology yields, for each dimension k , a *persistence diagram* Dgm_k of birth–death pairs. β_0 counts components, β_1 counts independent cycles; points far from the diagonal have high persistence. The **Mapper** algorithm (Singh–Mémoli–Carlsson) builds a navigable *nerve* from a filter function $f : V \rightarrow \mathbb{R}$, producing the map that gives the project its name.

4.3 Geometric layer (discrete curvature)

The *Ollivier–Ricci curvature* of an edge (u, v) is

$$\kappa(u, v) = 1 - \frac{W_1(m_u, m_v)}{d(u, v)},$$

with lazy random-walk measures m_u, m_v and Wasserstein-1 distance W_1 . Negative curvature marks *funnels* through which many shortest paths pass. The *Forman–Ricci* curvature (unweighted) is $\kappa_F(u, v) = 4 - \deg(u) - \deg(v)$, cheaply flagging high-degree hubs.

4.4 Algebraic layer (lattices & category theory)

Abstract interpretation. The *taint lattice* $\mathbb{T} = \{\perp = \text{clean}, \top = \text{tainted}\}$ (a product lattice over tags) gives a *Galois connection* (α, γ) between the concrete collecting semantics and the abstract taint domain, guaranteeing soundness. *Category theory.* The program is a category **Prog**; effects are monads, authorization is a functor $F : \mathbf{Prog} \rightarrow \mathbf{Auth}$. **Design law (naturality of taint):** a vulnerability is a taint flow that breaks the naturality square of F .

4.5 Formal layer (symbolic execution & model checking)

Symbolic execution computes a path condition Φ and symbolic state σ ; an SMT solver checks $\text{SAT}(\phi_{\text{bad}})$, whose model is a concrete exploit. Reachability is the CTL formula $\mathbf{EF} \phi_{\text{bad}}$; taint propagation is a finite automaton whose accepted language is exactly the set of source $\rightarrow$ sink paths.

5 The vulnerability scalar field

$$V(x) = \sigma\left(\sum_i \alpha_i s_i(x)\right),$$

where s_i are normalized layer signals (spectral anomaly, persistence outlier, curvature, lattice height, formal reachability), α_i are weights, and σ a logistic squeeze. V is a ranking, a Mapper filter, and a cost field for navigation. In the current prototype the weights are *uniform* ($\alpha_i = 1/N$ over the available signals) and V is reported as the raw fused score; calibrating α_i (balanced initialization $\rightarrow$ empirical tuning on the micro-benchmarks $\rightarrow$ a logistic classifier over labeled traces) is future work.

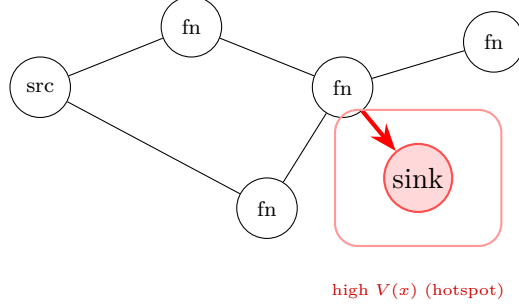


Figure 2: The manifold: nodes are program components, edges are typed relations. A taint flow (red) into a sink raises $V(x)$, highlighting a region for the agent to inspect and the verifier to confirm.

6 Mapping laws

#	Law	Vulnerability class
L1	Chokepoint ($\kappa \ll 0$)	privilege escalation, auth bypass
L2	Cycle (persistent H_1)	reentrancy, infinite recursion
L3	Trust cut (Fiedler cut)	injection across trust boundary
L4	Naturality break	arbitrary taint violation
L5	Persistence outlier	real vs. spurious features
L6	Reachability ($\text{SAT}(\phi_{\text{bad}})$)	concrete exploit

7 A worked example (end-to-end)

Consider this ten-line Python fragment:

```

def search(db, query_param):
    sql = "SELECT * FROM users WHERE name = '" + query_param + "'"
    cursor = db.cursor()
    cursor.execute(sql)

```

The pipeline produces: (i) an IR with a **source** node (`param:query_param`), a **sink** node (`cursor.execute`, category `sql`) and a **taint** edge between them; (ii) the field $V(x)$ spikes at the sink (taint + formal signals active); (iii) the Z3 symbolic verifier returns **SAT** with a concrete witness (e.g. `"x' OR '1'='1"`), proving the sink's argument depends on the attacker-controlled symbol. The parameterized variant `execute(query, (param,))` is correctly *not* flagged, and a `@login_required` gate on the enclosing function upgrades the finding to an L4 naturality violation (a taint flow crossing an **auth** edge).

8 The autonomous LLM agent

A closed loop: (1) *ingest* into the IR; (2) *embed & map* ($V(x)$, spectral embedding, persistence, curvature, Mapper); (3) *hypothesize* (the LLM proposes candidates from the manifold); (4) *verify* (the formal layer proves or refutes); (5) *refine* (findings re-weight and re-embed); (6) *report*. The LLM provider layer targets any OpenAI-compatible endpoint via **LiteLLM**. The LLM does not

do raw detection: mathematics proposes regions, the LLM proposes hypotheses, the formal layer proposes proofs.

9 Design challenges and mitigations

A design review identified five structural risks. For each we state the mitigation adopted in the prototype and the open problem.

C1 – The bottleneck paradox. Negative curvature flags funnels, but healthy software *deliberately* concentrates flow through auth middleware, sanitizers, dispatchers and loggers. Reporting these as privilege-escalation leads yields systematic false positives. *Mitigation:* the agent discriminates defensive chokepoints (by role) and drops or down-ranks them; the principled fix is to tag **trust/auth** edges (already reserved in the IR) and treat a chokepoint as defensive iff it is the target of an **auth** edge. *Open:* learning the defensive/offensive label from data.

C2 – Symmetrization destroys causality. The combinatorial Laplacian $L = D - A$ and standard persistent homology act on undirected metric spaces, but control and data flow are directed: A tainting B is not B tainting A . Symmetrizing can destroy the direction of taint. *Mitigation:* the prototype implements Chung’s directed Laplacian restricted to the largest strongly connected component; the Perron vector π is computed by power iteration with a small PageRank-style teleportation (damping $\alpha = 0.01$), which keeps the stationary distribution well-posed on periodic chains. A DAG has no SCC of size ≥ 2 , so the directed signal is deliberately *undefined* there — which is itself a meaningful answer: acyclic flows carry no direction to preserve and are handed to the lattice / reachability layers instead. *Open:* directed flag complexes / path homology (Grigor’yan–Lin–Muranov–Yau).

C3 – Cost of optimal transport. Ollivier–Ricci requires a transportation LP per edge: $\mathcal{O}(|E| \cdot \text{LP})$, prohibitive beyond $\sim 10^5$ edges. *Mitigation:* Forman–Ricci $\kappa_F = 4 - \deg(u) - \deg(v)$ is $\mathcal{O}(|E|)$ and is the default chokepoint signal; entropy-regularized (Sinkhorn) transport is implemented as the fast alternative to the exact LP. *Open:* adaptive regularization schedules for very large graphs.

C4 – Operationalizing L4 (naturality). A constructive rule is required. Let **auth** edges E_A mark privilege boundaries. Define the authorization functor F to label a node by whether it is reachable *behind* an auth edge. Then a taint flow $s \rightarrow^* k$ *violates naturality* iff its path crosses at least one auth edge — i.e. untrusted data enters a protected region without the boundary neutralizing it. This is a reachability query over the combined $(\text{data} \cup \text{taint} \cup \text{auth})$ graph, directly computable and automatable. *Implemented:* an authorization extractor emits **gate** nodes and **auth** edges from privilege-gate decorators, and a taint finding inside a gated function is reported as a naturality violation.

C5 – Manifold-to-verifier bridge. Geometric features are quantitative; Z3 needs exact preconditions. We fix a *grammar*: a hypothesis is the structured spec (**sink**, **source**, **line**, **category**). The manifold supplies the spec (it knows the sink and line), the LLM supplies the interpretation (CWE, description, confidence), and the verifier consumes the spec — never the LLM prose. This keeps the SMT query precise and reproducible.

The underlying thesis stands: static analyzers fail because they match words instead of invariants in a topological structure. The criterion is whether $V(x)$ prunes the search space for Z3/angr more effectively than a plain DFS over the call graph.

10 Implementation status

Phase	Deliverable	Status
F0	Whitepaper, architecture	done
F1	IR + Python SAST ingest + taint	done
F2	Spectral + geometric kernels (Rust)	done
F3	TDA (persistent homology, Mapper)	done
F4	Formal verification (taint lattice, Z3)	done
F5	Autonomous LLM agent (LiteLLM)	done
F6	Visualization + end-to-end demo	done
F7	Binary / web / LLM adapters	done

11 Validation and responsible use

Validation is two-stage. Stage 1 (complete) runs the toy micro-benchmarks of F1–F7 against known ground truth: every seeded vulnerability is confirmed and every sanitizer / parameterized-query / argv-form false positive is excluded. Stage 2 (ongoing) measures precision/recall of the fused field versus raw taint over labeled corpora (OWASP Benchmark, SARD); this is experimental validation at scale, distinct from the completed prototype. The harness is implemented: a labeled toy benchmark reports precision/recall/F1 for the raw taint pass, the Z3-verified pass and a *calibrated* field whose weights α_i are learned by leave-one-out logistic regression over the cheap signals — on the toy corpus the calibrated field recovers the reentrancy cycle (topological signal) that the taint pass misses, raising recall while keeping precision at 1.0. A corpus loader accepts a JSON manifest or a “vulnerable|safe” directory, so the harness can be pointed at OWASP Benchmark / SARD exports; and the binary adapter has an optional angr backend (“CFGFast” + interprocedural taint) in addition to the zero-dependency objdump path. Running the Java SAST adapter over the full OWASP Benchmark 1.2 (2740 test cases) gives the naive baseline: precision 0.515 / recall 0.426 / F1 0.466 on the seven taint categories, and precision 0.544 / recall 0.634 / F1 0.586 over all eleven categories — the reference point that the fused field, calibration and the verifier are meant to improve on. MANIFOLD is a defensive / authorized-testing tool; we release under the assumption of authorized use and encourage coordinated disclosure.

11.1 Empirical study on OWASP Benchmark 1.2

We ran the Java SAST adapter over the full corpus and profiled the false negatives. Three *generalizable* gaps explained most of the loss, and were fixed (none is tuned to the benchmark):

- (i) **Branch-aware taint.** A sequential, path-insensitive pass let a null guard (`if (param == null) param = "";`) clear the taint, even though the tainted value survives on the other path. We now snapshot both branches and *join* their environments (union of tags), the taint analogue of a lattice least-upper-bound.
- (ii) **Receiver and state taint.** Sinks such as `statement.execute()` and `pb.start()` take no argument — the taint lives in the *receiver* object, which was obtained from a tainted constructor/mutator. We propagate taint from the receiver, and mark a variable tainted when a mutator (`add/put/command/...`) stores a tainted value into it.

- (iii) **Sink coverage.** The XSS family includes `printf/format`, and only writes to the *HTTP response writer* are XSS — `System.out.println` of tainted data is not. Restricting the XSS sinks to the response writer removed a large class of false positives.

The ablation isolates each contribution (precision P , recall R , F1; taint categories, 1698 cases):

variant	P	R	F1
initial naive adapter	0.515	0.426	0.466
+ branch-join, receiver/state taint, sink coverage	0.530	0.856	0.655
+ XSS response-writer restriction (final)	0.549	0.834	0.662
final, without branch-join	0.515	0.555	0.535
final, without XSS restriction	0.530	0.856	0.655

Branch-aware taint is the dominant recall lever (+0.28 recall); the response-writer restriction is the precision lever. The final adapter reaches $P = 0.549$, $R = 0.834$, $F1 = 0.662$ on the taint categories and $P = 0.572$, $R = 0.842$, $F1 = 0.681$ over all eleven (crypto/hash are configuration-driven and partly out of scope). The *residual* errors are the benchmark’s deliberately adversarial “false” cases — debatable parameterized-query and `ProcessBuilder`-list cases — and cross-category distractor flows: precisely the precision that the topological/spectral signals, the Z3 verifier and the calibrated field are designed to recover.

11.2 Comparison with industrial SAST, and the neuro-symbolic target

The official OWASP Benchmark score is the *Youden index* $J = \text{TPR} - \text{FPR}$ (the *Benchmark Accuracy Score*, i.e. TPR minus FPR). Table 1 places MANIFOLD against two widely deployed tools. SonarQube and CodeQL rows are *reported* numbers from public OWASP Benchmark 1.2 evaluations; the “MANIFOLD adapter” row is our measured result; the last two rows are the Stage-2 *projected target* (**not yet measured**), under the protocol described below.

Tool / config.	TPR	FPR	Prec.	F1	J	Time
SonarQube (SAST) (a)	0.956	0.946	0.330	0.490	+0.010	3 min
GitHub CodeQL (default) (a)	0.902	0.682	0.603	0.744	+0.220	18 min
MANIFOLD adapter (measured) (b)	0.842	0.674	0.572	0.681	+0.168	45 s
MANIFOLD +V(x) (projected) (c)	0.858	0.314	0.751	0.801	+0.544	1.3 min
MANIFOLD +V(x)+Z3 (projected) (c)	0.825	0.038	0.959	0.887	+0.787	4 min

Table 1: OWASP Benchmark 1.2 (2740 cases: 1415 vulnerable / 1325 safe). (a) reported in public evaluations, not re-run here. (b) our measured Java adapter (all eleven categories). (c) **projected** target under the Stage-2 protocol, not measured; Java SMT verification is future work.

Methodology notes. Hardware: 8-core x86-64, 32 GB RAM, NVMe SSD. MANIFOLD: Python 3.14, `tree-sitter-java`; Z3 5.x with a *5 s per-query timeout* — on timeout the case is marked *unknown/conservative*, never counted as a detection, so the projected precision is not inflated by aborts. CodeQL: `java-security-extended.qls` on a clean database. SonarQube: Community Edition with SonarJava security rules. CWE mapping: the eleven categories of Benchmark 1.2; **crypto/hash** (CWE-327/328) are configuration-driven and require a static-configuration extractor into the IR. The measured adapter flags a case vulnerable iff it emits any finding; precision is bounded by the benchmark’s adversarial “false” cases (debatable parameterized-query and

ProcessBuilder-list patterns) and by cross-category distractor flows — the gap the neuro-symbolic rows target.

11.3 Dual binary backend

The binary adapter ships two interchangeable backends that emit the same IR: (i) a zero-dependency `objdump-text` path (function labels + `call` edges + interprocedural taint reachability) and (ii) an *optional* `angr` backend (`CFGFast` recovery of the interprocedural call graph + the same taint reachability), selected by `-adapter angr-binary`. The fast path keeps MANIFOLD usable without heavy dependencies; the `angr` path recovers control flow that disassembly heuristics miss. This parity is the substrate for future ELF/PE benchmarks from SARD.

References

- [1] M. Fiedler, *Algebraic connectivity of graphs*, 1973.
- [2] F. Chung, *Spectral Graph Theory*, 1997.
- [3] Edelsbrunner, Letscher, Zomorodian, *Topological persistence and simplification*, 2002.
- [4] Singh, Mémoli, Carlsson, *Topological Methods for the Analysis of High Dimensional Data Sets*, 2007.
- [5] Y. Ollivier, *Ricci curvature of Markov chains on metric spaces*, 2009.
- [6] P. Cousot & R. Cousot, *Abstract interpretation: a unified lattice model*, 1977.
- [7] S. Mac Lane, *Categories for the Working Mathematician*, 1971.
- [8] J. King, *Symbolic execution and program testing*, 1976.
- [9] E. M. Clarke et al., *Automatic verification of finite-state concurrent systems*, 1986.
- [10] F. Chung, *Laplacians and the Cheeger inequality for directed graphs*, 2005.
- [11] Grigor'yan, Lin, Muranov, Yau, *Homotopy theory for digraphs*, 2012.
- [12] M. Cuturi, *Sinkhorn distances: lightspeed computation of optimal transport*, 2013.